Java main method breakdown

Java main method explanation in details each word

In Java, the `main` **method** is the **entry point of a Java program**. When you run a Java application, the **JVM (Java Virtual Machine)** looks for this method first and starts executing the program from here.

Basic syntax:

```Java
public static void main String   args
    // program starts here
```

Now let's understand **each word in detail**.

---

1. `public`

`public` is an **access modifier**.

Meaning

It means the method is **accessible from anywhere**.

Why `public` is required?

The **JVM is outside your class**, so it needs permission to call the `main` method.
If `main` were `private` or `protected`, the JVM **would not be able to access it**.

Example (Wrong):

```Java
private static void main String   args
```

This will give an error because **JVM cannot access private methods**.

Simple understanding

`public` → JVM can access and execute this method.

---

2. `static`

`static` means the method **belongs to the class, not to objects**.

Why `static` is required?

When a program starts, **no objects of the class exist yet**.

So the JVM cannot do:

```java
Test obj = new Test
obj.main
```

Instead, the JVM calls the method **directly using the class name**.

Example:

```java
Test.main
```

This is possible **only if the method is static**.

Simple understanding

Without static	With static
Object required	No object required
`obj.method()`	`Class.method()`

So JVM can directly run:

[redacted]

---

3. `void`

`void` is the **return type**.

It means the method **does not return any value**.

Example:

```Java
int add int a  int b
```

This method returns an integer.

But `main` does **not return anything to the JVM**, so its return type is:

[redacted]

Example:

```Java
public static void main String   args
```

Meaning:

[redacted]

---

4. `main`

`main` is the **name of the method**.

The **JVM specifically looks for a method named** `main`.

If the name is changed, the program will **not run**.

Example (Wrong):

```Java
public static void start String   args
```

Error:

[redacted]

Because JVM searches specifically for:

[redacted]

Simple understanding

`main` → **starting point of program execution**

---

5. `String[]`

`String[]` means **array of String objects**.

It stores **command-line arguments**.

Example when running program:

[redacted]

Then:

[redacted]

So the JVM passes these values to the program.

Example code:

Java

```
public class Test
    public static void main String   args

        System.out.println args 0
        System.out.println args 1
```

Output:

[redacted]

Simple understanding

`String[]` → multiple string inputs from command line.

---

6. `args`

`args` is the **name of the array variable**.

It stores the **command-line arguments**.

Example:

```java
String   args
```

You can also change its name.

Example:

```java
public static void main String   input
```

or

```java
public static void main String   data
```

These are also valid.

But `args` is used **by convention**.

Complete Meaning of Main Method

```
```

Meaning:

Word	Meaning
public	JVM can access this method
static	JVM can call it without creating object
void	method does not return anything

Word	Meaning
main	starting point of Java program
String[]	array of command line inputs
args	variable storing those inputs

Example Program

Java

```java
public class Hello

    public static void main String   args

        System.out.println "Hello World"
```

Execution flow:

Important Interview Points

1️⃣ Valid main method signatures

All these are valid:

```Java
public static void main String   args
public static void main String args
public static void main String    args
```

2️⃣ Order of `public static` can change

Valid:

```Java
static public void main String   args
```

3️⃣ Main method can be overloaded

Example:

```Java
public class Test

    public static void main String   args
        main 10


    public static void main int a
        System.out.println a
```

But JVM always calls:

Simple Real-Life Analogy

Think of Java program like a **movie**.

The director always starts the movie from the **first scene (main method)**.

If you want, I can also explain **advanced concepts of main method**, like:

- Why `String` is used instead of other data types
- How JVM actually calls `main()` internally
- What happens **inside JVM when main method runs**
- Why `main` is **static in JVM design**

These are **very important for interviews**.

Explain System.out.println() in details

`System.out.println()` is one of the most commonly used statements in Java to **print output to the console (screen)**.

Example:

```Java
System.out.println "Hello World"
```

Now let's **break it down word by word** so you clearly understand how it works internally.

---

1. `System`

`System` is a **predefined class in Java**.

It belongs to the package:

Since `java.lang` is **automatically imported**, you do not need to write:

```Java
import java.lang.System
```

Purpose of `System` class

The `System` class provides access to **system-level features**, such as:

- Standard input
- Standard output
- Standard error
- Current time
- Garbage collection
- Environment variables

Example methods/fields:

```java
System.out
System
System.err
System currentTimeMillis
System.gc
```

Simple meaning

---

2. `out`

`out` is a **static variable** inside the `System` class.

Its declaration inside Java is approximately like this:

```java
public static final PrintStream out
```

So:

- `public` → accessible everywhere
- `static` → belongs to the class
- `final` → cannot be reassigned
- `PrintStream` → type of the object

What does `out` represent?

`out` represents the **standard output stream**, which is normally the **console (terminal)**.

So when you write:

```java
System.out
```

You are accessing the **PrintStream object responsible for printing output**.

Visual idea

[redacted]

---

3. `println`

`println` is a **method** of the `PrintStream` class.

It prints the value and **moves the cursor to the next line**.

Meaning of the name

[redacted]

So:

[redacted]

Example:

```java
System.out.println "Hello"
System.out.println "World"
```

Output:

[redacted]

4. `()` (Parentheses)

Parentheses contain the **argument** that you want to print.

Example:

```java
System.out.println "Hello"
System.out.println 10
System.out.println 5 + 3
```

The value inside the parentheses is sent to the `println()` method.

---

5. `;` (Semicolon)

The semicolon **terminates the statement**.

Java requires every statement to end with:

```
```

Example:

```java
System.out.println "Hello"
```

Without `;` the program will give a **compile-time error**.

Internal Structure of `System.out.println()`

The statement actually works like this:

```java
System.out.println "Hello"
```

Step-by-step execution:

```
```

Flow:

Simplified Internal Representation

It roughly behaves like this:

```java
class System
    public static PrintStream out = new PrintStream
```

And inside `PrintStream`:

```java
class PrintStream

    void println String x
        print x
        newLine
```

So when you write:

```java
System.out.println "Hello"
```

It means:

```
```

Difference Between `print` and `println`

Method	Behavior
`print()`	prints text but stays on same line
`println()`	prints text and moves to next line

Example:

```java
System.out.print "Hello "
System.out.print "World"
```

Output:

```
```

Example:

```java
System.out.println "Hello"
System.out.println "World"
```

Output:

```
```

Types of Data `println()` Can Print

`println()` is **overloaded**, meaning it has multiple versions.

It can print:

String

```java
System.out.println "Java"
```

Integer

```java
System.out.println 10
```

Double

```java
System.out.println 10.5
```

Boolean

```java
System.out.println true
```

Character

```java
System.out.println 'A'
```

Object

```java
System.out.println object
```

Real-Life Analogy

Think of it like this:

So:

means:

Important Interview Point

Full interpretation:

So the complete meaning is:

If you want, I can also explain **some deeper concepts** that many students don't know but are **very useful for interviews**, like:

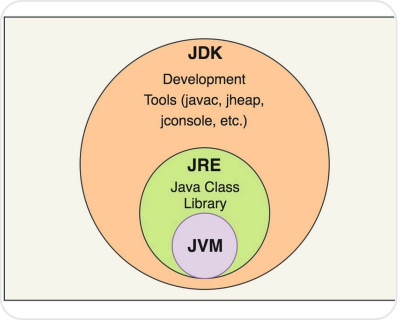
- Why `System.out` is **static**
- Why `PrintStream` is used instead of `PrintWriter`
- How **Java internally prints text to the console**
- What happens **inside JVM when println() is called**.

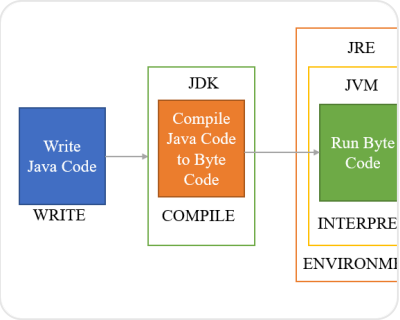
Explain JVM, JDK and jre in detailed manner with figures.

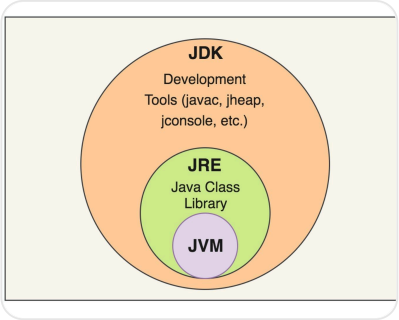
To understand **Java execution**, you must clearly understand **JDK, JRE, and JVM** and how they work together. Think of them as **layers of the Java platform**.

---

1. JDK (Java Development Kit)







Definition

JDK (Java Development Kit) is the **complete package used to develop Java applications**.

It contains **tools required for writing, compiling, debugging, and running Java programs**.

Simple Meaning

[redacted]

So the **JDK is the largest component**.

Components of JDK

1️⃣ JRE (Java Runtime Environment)

Allows Java programs to run.

2️⃣ Development Tools

These help developers build programs.

Important tools inside JDK:

Tool	Purpose
`javac`	Compiles Java code into bytecode
`java`	Runs the compiled Java program

Tool	Purpose
`javadoc`	Generates documentation
`jar`	Packages Java files into JAR files
`jdb`	Java debugger
`javap`	Disassembles bytecode

Example:

[redacted]

Compiles the Java file into:

[redacted]

Structure of JDK

[redacted]

Role of JDK

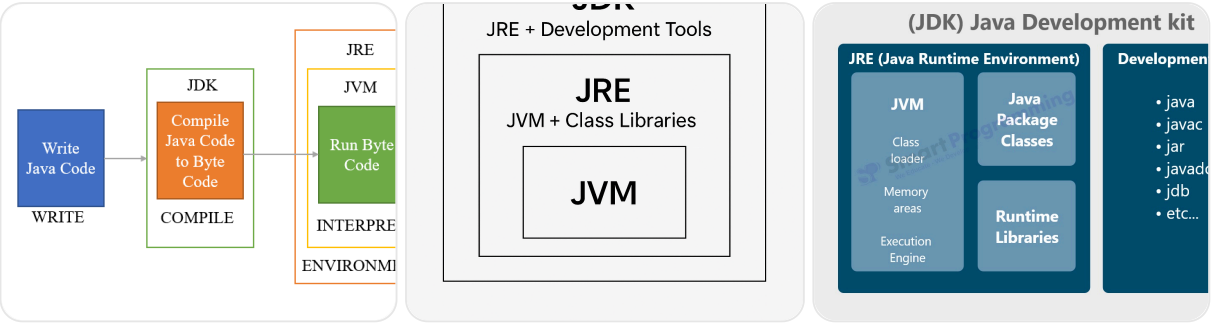
JDK is used for:

- Writing Java programs
- Compiling Java code

- Running Java applications
- Debugging programs
- Packaging applications

---

2. JRE (Java Runtime Environment)



Definition

JRE (Java Runtime Environment) provides the **environment required to run Java programs**.

It does **not include development tools**.

Simple Meaning

[redacted]

Components of JRE

1️⃣ JVM (Java Virtual Machine)

Executes Java bytecode.

2️⃣ Class Libraries

Prewritten Java classes used by programs.

Examples:

[redacted]

Example usage:

```java
System.out.println "Hello"
```

`System` belongs to:

which is part of **JRE libraries**.

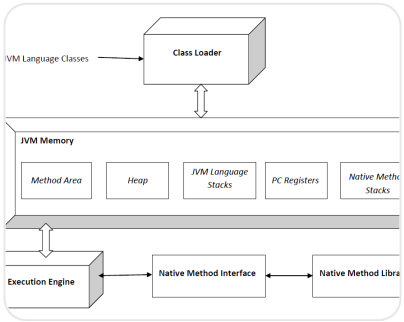
Role of JRE

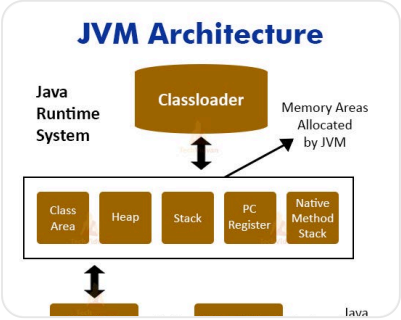
JRE is responsible for:

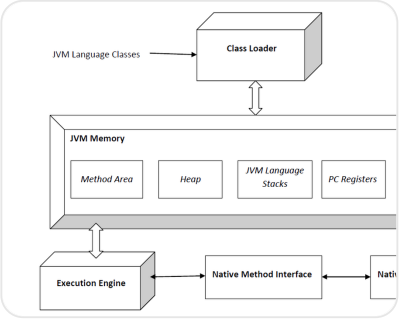
- Loading classes
- Verifying bytecode
- Running Java programs
- Providing runtime libraries
- Managing memory

---

3. JVM (Java Virtual Machine)







Definition

The **JVM (Java Virtual Machine)** is the **engine that runs Java programs**.

It converts **bytecode into machine code** and executes it.

Simple Meaning

[redacted]

How Java Program Runs (Step-by-Step)

Example program:

```Java
public class Hello
    public static void main String   args
        System.out.println "Hello World"
```

Step 1 — Write Java code

[redacted]

Step 2 — Compilation

Using compiler:

[redacted]

Output:

[redacted]

This `.class` file contains **bytecode**.

Step 3 — Execution

[redacted]

JVM loads and runs the bytecode.

Flow Diagram

[redacted]

Internal Architecture of JVM

JVM mainly contains **three major components**.

1️⃣ Class Loader

Loads `.class` files into memory.

Steps:

[redacted]

Example:

[redacted]

2️⃣ Runtime Data Areas (Memory Areas)

JVM uses several memory areas.

Method Area

Stores:

- class metadata
- static variables
- methods

Heap

Stores **objects and instance variables**.

Example:

```java
Student s = new Student
```

The object `s` is stored in **Heap memory**.

Stack

Stores:

- method calls
- local variables

Example:

```java
int a = 10
```

Stored in **stack memory**.

PC Register

Keeps track of **current executing instruction**.

Native Method Stack

Used when Java interacts with **native libraries (C/C++)**.

3️⃣ Execution Engine

Responsible for executing bytecode.

It has two parts.

Interpreter

Executes bytecode **line by line**.

But it is **slower**.

JIT Compiler (Just-In-Time Compiler)

Converts bytecode into **native machine code** for faster execution.

JVM Execution Flow

Relationship Between JDK, JRE, and JVM

[redacted]

Simple Real-Life Analogy

Think of Java like **a car factory**.

Component	Analogy
JDK	Car manufacturing factory
JRE	Engine system
JVM	Actual engine that runs the car

Comparison Table

Feature	JDK	JRE	JVM
Full form	Java Development Kit	Java Runtime Environment	Java Virtual Machine
Purpose	Development	Running programs	Executing bytecode
Contains	JRE + Tools	JVM + Libraries	Execution engine
Used by	Developers	End users	System

Important Interview Point

Most students remember this formula:

[redacted]

Therefore:

[redacted]

JDK is the **largest**, JVM is the **smallest**.

✅ Since you're a **Java backend learner and working with Spring Boot**, understanding this architecture helps you understand:

- how Java applications run
- how memory works
- how JVM tuning works in backend systems.

If you want, I can also explain **JVM Memory Management (Heap, Stack, Garbage Collector)** with **very clear diagrams**, which is one of the **most asked Java interview topics**.